

第七次作业报告

毛川 2300013218

代码解释

以下是主要的代码片段，包括 n-step Sarsa 的 Q 值更新和训练循环，代码中有所化简：

```

# n-step Sarsa Q-value update
def update_q(self, tau, T):
    # batch update for each episode in the buffer
    low = tau + 1
    high = min(tau + self.n, T)
    G = 0.0

    # Calculate the n-step return
    for i in range(low, high + 1):
        assert(i-1 < len(self.buffer)), f'Buffer length: {len(self.buffer)}, i-1: {i-1}, low:
        r_i = self.buffer[i-1][2] # reward r_i
        G += (self.gamma ** (i - tau - 1)) * r_i

    # bootstrapped term: only if tau + n < T
    if tau + self.n < T:
        s_tn = self.buffer[tau + self.n-1][3] # s_{t+n}
        a_tn = self.buffer[tau + self.n-1][4]
        # max over actions at s_{t+n}
        G += (self.gamma ** self.n) * self.q[s_tn][a_tn]

    # get (s_tau, a_tau)
    state_tau = self.buffer[tau][0]
    action_tau = self.buffer[tau][1]

    # perform Q update for the single (s_tau, a_tau)
    td_error = G - self.q[state_tau][action_tau]
    # Update Q-value
    self.q[state_tau][action_tau] += self.lr * td_error

# n-step Sarsa training loop
def train(self):
    self.env_step_num = 0

    for episode_num in range(self.start_iter, self.iter+1):
        T = 1e6 # Set to infinity
        state = self.env.reset()
        done = False
        action = self.epsilon_greedy(state)
        self.buffer = list()
        current_step = -1 # record current step in episode
        tau = 0 # time whose estimate is being updated

        while tau != T - 1:

```

```

current_step += 1
self.epsilon_decay(self.env_step_num)

if current_step < T: # not terminal
    # take action, observe new state and reward
    new_state, reward, done, _ = self.env.step(action)
    self.env_step_num += 1
    if done:
        # if episode ends, set T
        reward = self.end_reward
        T = current_step + 1 # for last s_{t+1} is terminal
        # print(f'Episode {i} ended at step {T-1}, last {current_step} steps.')
        self.add_to_buffer((state, action, reward, state, -1)) # only need state
    else:
        # not terminal, take next action
        new_action = self.epsilon_greedy(new_state)
        if self.render:
            self.env.render()
        self.add_to_buffer((state, action, reward, new_state, new_action)) # on

# update use tau
tau = current_step - self.n + 1
if tau >= 0:
    self.update_q(tau, T)
state = new_state
action = new_action if not done else -1

```

代码思路为：在每个 episode 中，使用一个 buffer 来存储 (state, action, reward, next_state, next_action) 五元组。积累的时间步数达到 n 时，调用 update_q 方法计算 n-step return G，并更新当前所属位置 n 步前的 state 的 Q 值。训练循环中，智能体与环境交互，收集数据并进行 Q 值更新，直到 episode 结束。在结束后，继续更新剩余的 Q 值直到所有时间步都被更新，此时并不为严格的 n-step。

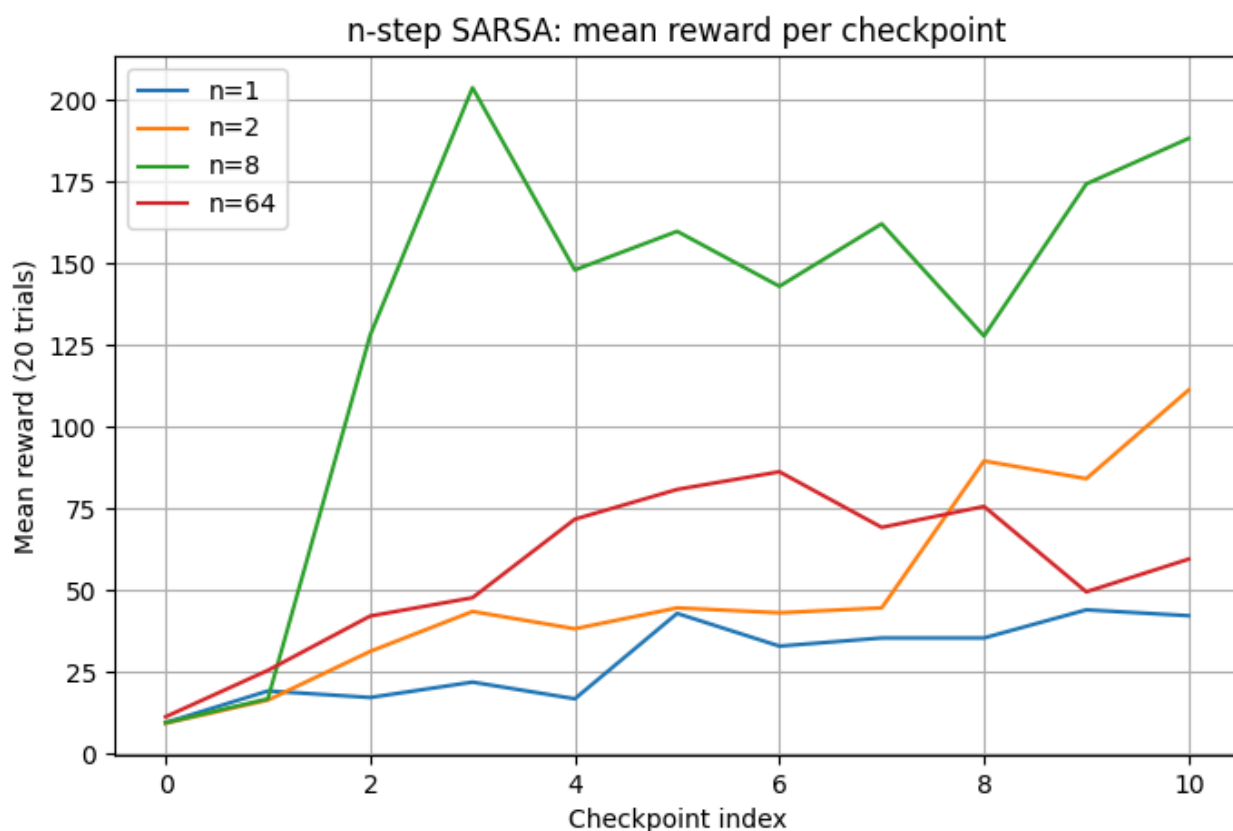
问题探究

1. 实验可靠性：interval = 8，导致训练不稳定，随机性较大，结果波动较大。解决方案：评测每个 checkpoint 多跑几次，取平均值。
2. 评测指标：在课本中对于 n-step estimate 中采用 α 作为横轴，10-episode Average RMS error 作为纵轴的图像进行对比实验。但在实际中，我们的任务是 n-step TD control，而非 prediction，因此采用训练时间为横轴，评测每个 checkpoint 下的平均回报作为纵轴，更符合实际任务需求。

3. 实验设计公平性：对于不同的 n 值，我们已知不同的 n 值下，学习率 α 的最优值不同。但是为每个 n 值都调参，工作量较大。因此我们选择了一个折中的方案：在开始大规模实验前，针对每个 n 值，调节学习率 α ，选择一个较优的 α 值用于后续大规模实验。这样虽然不能保证每个 n 值下的 α 都是最优的，但至少能保证每个 n 值下的 α 都是较优的，从而保证实验的公平性。
4. 迭代次数设置：在一般的 RL 任务中，为了比较两个方法的效果，通常控制不变的变量是与环境的交互次数，比较经过相同的交互次数时保存的 checkpoint 的平均期望回报。但是遇到的问题是不同的 n 值下，相同 step 数下 reward 接近。反思后发现这是基于 cartpole 对于一个较好的策略，在一个 episode 中 step 数目更多，而这些 step 往往维持在特定的 state 范围内，导致 reward 接近，学习效率低。因此作出调整，比较的是相同的 episode 数下保存的 checkpoint 的平均期望回报。给予这些 agent 相同的尝试机会，从而更公平地比较不同 n 值下的效果。经过实验，得到了有差异的结果。

实验结果及分析

以下是不同 n 值下，训练过程中平均回报的变化曲线：在 1000 个 episode 中，每隔 100 个 episode 保存 q table，并再最终统一评测每个 checkpoint 下的平均回报，评测 20 次，去掉最高的 5 次和最低的 5 次，取中间 10 次评测的平均值作为该 checkpoint 的最终评测结果。



在探究的 4 种 n 的取值中， $n=4$ 的表现最佳，其次是 $n=2$ ， $n=1$ 和 $n=8$ 的表现相对较差。分析原因如下：

1. $n=1$ 时，等价于传统的 Sarsa 方法，更新时仅考虑了当前时间步的奖励和下一个时间步的估计值，信息利用较少，导致学习速度较慢。在 1000 个 episode 内，智能体难以充分学习到有效的策略，因此表现较差。
2. $n=2$ 时效果优于 $n=1$ ，因为它利用了更多的未来奖励信息，能够更快地调整 Q 值，从而提升学习效率。
3. $n=8$ 时效果最佳，可能是因为在 CartPole 环境中，考虑到相应的状态规模和当前设置的其他超参数， $n=8$ 能够更好地平衡偏差和方差。虽然有所波动，但是达到了最高的平均回报。
4. $n=64$ ，当 n 过大时，虽然考虑了更多的未来奖励，但也引入了较大的方差，期间(300-700 episode)的表现好于 $n=1$ 和 $n=2$ ，但之后的表现有所下降。